

# PYTHON WEEK 4 TEST: OOP

Sections: MCQ (40), Predict Output (55), Quick Refactor (5).

## Section A: MCQ (1-40)

**Q1. When should inheritance be preferred over composition?**

- A. When we want fewer lines of code
- B. When there is a true and stable `is-a` relationship
- C. When runtime dependency swapping is required
- D. When we cannot use constructors

**Q2. `super()` in Python resolves methods using:**

- A. Direct parent only
- B. MRO chain
- C. Alphabetical class order
- D. Import order

**Q3. Which principle is broken if child class rejects valid parent inputs?**

- A. DRY
- B. SRP
- C. LSP
- D. KISS

**Q4. `typing.Protocol` is mainly used for:**

- A. Mandatory inheritance at runtime
- B. Structural typing contracts
- C. Faster execution
- D. Constructor chaining

**Q5. MRO in Python ensures:**

- A. No need for tests
- B. Deterministic method lookup
- C. Automatic deep copy
- D. Parent constructors run twice

**Q6. Which is a fragile base class smell?**

- A. Small stable parent API
- B. Parent edits frequently break many children
- C. Abstract contract with focused methods
- D. Strong contract tests

**Q7. Best design for pluggable payment providers:**

- A. `if/elif` everywhere
- B. Payment abstraction + implementations
- C. Global provider string checks
- D. Duplicate classes per module

**Q8. Why is composition often easier to test?**

- A. No classes needed
- B. Dependencies can be injected/faked
- C. It removes interfaces
- D. It avoids constructors

**Q9. Deep hierarchy (`A->B->C->D->E`) with frequent breakage should first be:**

- A. Extended with more children
- B. Flattened and behavior extracted into composition
- C. Replaced with globals

D. Ignored

**Q10. LSP-safe child should:**

- A. Keep compatible semantics with parent contract
- B. Always return different type
- C. Always throw more exceptions
- D. Always skip parent logic

**Q11. Best use of mixin:**

- A. Full domain model with heavy state
- B. Small reusable cross-cutting behavior
- C. Replacing strategy pattern always
- D. Database access layer

**Q12. Lower coupling in service layer usually means:**

- A. Hardcoded concrete classes in constructor
- B. Depend on abstractions via injection
- C. Runtime `eval` for dependencies
- D. Multiple inheritance for all services

**Q13. Strong sign polymorphism is missing:**

- A. Abstraction with many implementations
- B. Repeated `instance` behavior branching
- C. Contract test suite
- D. Strategy objects

**Q14. "Prefer composition over inheritance" mostly optimizes for:**

- A. Deeper class trees
- B. Change isolation and flexibility
- C. Fewer tests
- D. No abstractions

**Q15. Correct statement about ABC vs Protocol:**

- A. Both require explicit subclassing
- B. ABC is explicit inheritance contract; Protocol is structural contract
- C. Protocol enforces runtime inheritance
- D. ABC cannot contain implemented methods

**Q16. Best constructor rule in extensible parents:**

- A. Call overridable methods before initialization
- B. Keep constructor minimal and avoid overridable calls
- C. Access child internals directly
- D. Force abstract constructor

**Q17. Cooperative multiple inheritance requires:**

- A. Direct parent class calls only
- B. Consistent `super()` usage in chain
- C. No constructors
- D. Private methods only

**Q18. OCP-friendly extension for new report format:**

- A. Add another `elif`
- B. Add new implementation class for same contract
- C. Replace old report class entirely
- D. Hardcode format in controller

**Q19. Which is NOT an LSP violation?**

- A. Child preserves contract and semantics
- B. Child rejects valid parent input
- C. Child returns incompatible behavior
- D. Child tightens mandatory preconditions

**Q20. Why inheritance for utility reuse is usually bad:**

- A. Python disallows it
- B. It creates fake `is-a` relation and semantic confusion
- C. It always slows code
- D. It breaks typing tools

**Q21. Runtime logger swap is best done by:**

- A. Giant logger hierarchy only
- B. Composed logger strategy
- C. Global print patching
- D. Inheriting all services from logger

**Q22. First question before introducing inheritance:**

- A. "Will this reduce file count?"
- B. "Does subclass truly satisfy parent identity and contract?"
- C. "Can I avoid tests?"
- D. "Can I avoid naming?"

**Q23. Contract tests for polymorphic classes are useful because they:**

- A. Remove all integration tests
- B. Verify common behavior expectations across implementations
- C. Validate comments only
- D. Make runtime errors impossible

**Q24. Plugin-friendly architecture usually favors:**

- A. One giant base class with many optional methods
- B. Narrow contracts + composition + registration
- C. Monkeypatching core flow
- D. Hardcoded plugin list in many files

**Q25. For `OrderService` using payment+discount+notifier:**

- A. Inherit all three
- B. Compose/inject all three
- C. Make them global
- D. Duplicate service classes

**Q26. Most direct OCP refactor:**

- A. Rename methods
- B. Replace type-branching with polymorphic dispatch
- C. Add TODO comments
- D. Inline abstractions

**Q27. Overusing inheritance often causes:**

- A. Lower coupling
- B. Ripple regressions from base changes
- C. Better local reasoning
- D. Simpler runtime behavior

**Q28. Correct HAS-A design:**

- A. ``class Car(Engine)``
- B. ``class Car: self.engine = engine``

C. ``class Engine(Car)``

D. ``class Car(ABC)``

**Q29. Behavior variation independent of hierarchy is best modeled with:**

- A. More subclasses
- B. Strategy composition
- C. Class variables
- D. Globals

**Q30. Best interview explanation:**

- A. Composition always replaces inheritance
- B. Start composition-first; use inheritance for stable true ``is-a``
- C. MRO is rarely practical
- D. LSP is optional

**Q31. Which ``super()`` statement is true in multiple inheritance?**

- A. It skips intermediate classes
- B. It follows next class in MRO, not direct parent assumption
- C. It calls all classes automatically without overrides
- D. It works only in ``__init__``

**Q32. What is strongest reason to keep parent APIs minimal?**

- A. Smaller parent APIs reduce fragility and coupling
- B. Fewer docs needed
- C. Better runtime speed guaranteed
- D. Eliminates inheritance bugs

**Q33. Child override should keep exception behavior:**

- A. completely unrelated to parent expectations
- B. compatible and predictable for callers
- C. random for each child
- D. hidden

**Q34. In dependency injection, high-level module should depend on:**

- A. concrete classes
- B. abstractions/contracts
- C. global state
- D. static helper methods only

**Q35. Most accurate statement about duck typing:**

- A. It requires base class inheritance
- B. It accepts objects by behavior compatibility
- C. It disables type checking tools
- D. It is not polymorphism

**Q36. Which is better for quickly adding new discount logic?**

- A. Edit large checkout method repeatedly
- B. Add new ``DiscountStrategy`` implementation
- C. Duplicate checkout service
- D. Modify entity constructors

**Q37. When should you add MRO-specific unit tests?**

- A. Never
- B. When multiple inheritance affects production-critical behavior
- C. Only for toy examples
- D. Only for static methods

**Q38. Which is cleanest rule for base contracts?**

- A. Hidden assumptions in code only
- B. Explicit preconditions/postconditions and consistent semantics
- C. No exception policy
- D. Optional return type shifts

**Q39. Most correct fix for class-level mutable bug:**

- A. Keep list at class level for all instance data
- B. Move mutable field initialization to `__init__`
- C. Use tuple then mutate
- D. Use global list

**Q40. Best quick architecture for extensible notifications:**

- A. giant `if channel == ...`
- B. `Notifier` contract + implementations + injected usage
- C. one method per channel in service
- D. subclass service for each channel

**Section B: Predict the Output (41-95)**

**Q41. Predict the output:**

```
class Base:
    def label(self):
        return "base"

class Child(Base):
    pass

print(Child().label())
```

Answer:

---

**Q42. Predict the output:**

```
class Account:
    def __init__(self, amount):
        self.amount = amount

class Premium(Account):
    def __init__(self, amount):
        super().__init__(amount + 25)

print(Premium(75).amount)
```

Answer:

---

**Q43. Predict the output:**

```
class A:
    def who(self):
        return "A"

class B(A):
    def who(self):
        return "B>" + super().who()

print(B().who())
```

Answer:

---

**Q44. Predict the output:**

```
class A: pass
class B(A): pass
class C(B): pass
print([c.__name__ for c in C.mro()])
```

Answer:

---

**Q45. Predict the output:**

```
class P:
    level = "P"

class Q(P):
    pass

q1 = Q()
q2 = Q()
q1.level = "Q1"
print(q1.level, q2.level, P.level)
```

Answer:

---

**Q46. Predict the output:**

```
class X:
    def __init__(self):
        self.__k = 9

x = X()
print("_X_k" in x.__dict__)
```

Answer:

---

**Q47. Predict the output:**

```
class Logger:
    def log(self, m):
        print("INFO", m)

class Audit(Logger):
    def log(self, m):
        super().log(m)
        print("AUDIT", m)

Audit().log("ok")
```

Answer:

---

**Q48. Predict the output:**

```
from abc import ABC, abstractmethod

class Job(ABC):
    @abstractmethod
    def run(self):
        pass

class Worker(Job):
    def run(self):
        return "done"

print(Worker().run())
```

Answer:

---

**Q49. Predict the output:**

```
from abc import ABC, abstractmethod

class Job(ABC):
    @abstractmethod
    def run(self):
        pass

class Broken(Job):
    pass

Broken()
```

Answer:

---

**Q50. Predict the output:**

```
from typing import Protocol

class CanSend(Protocol):
    def send(self, m: str) -> None: ...

class Push:
    def send(self, m: str) -> None:
        print("PUSH", m)

def emit(ch: CanSend):
    ch.send("deploy")

emit(Push())
```

Answer:

---

**Q51. Predict the output:**

```
class Email:
    def send(self, m):
        print("EMAIL", m)

class SMS:
    def send(self, m):
        print("SMS", m)

def notify(ch, m):
    ch.send(m)

notify(Email(), "x")
notify(SMS(), "y")
```

Answer:

---

**Q52. Predict the output:**

```
class Engine:
    def start(self):
        return "engine"

class Car:
    def __init__(self, engine):
        self.engine = engine

    def start(self):
        return "car-" + self.engine.start()

print(Car(Engine()).start())
```

Answer:

---

**Q53. Predict the output:**

```
class D0:
    def apply(self, t):
        return t

class D20:
    def apply(self, t):
        return t * 0.8

class Bill:
    def __init__(self, d):
        self.d = d

    def total(self, t):
        return self.d.apply(t)

print(Bill(D0()).total(500))
print(Bill(D20()).total(500))
```

Answer:

---

**Q54. Predict the output:**

```
class JsonLogger:
    def log(self, m):
        print(f'{{"m": "{m}"}}')

class Service:
    def __init__(self, logger):
        self.logger = logger

    def run(self):
        self.logger.log("go")

Service(JsonLogger()).run()
```

Answer:

---

**Q55. Predict the output:**

```
class P: pass
class C(P): pass
print(isinstance(C(), P), issubclass(C, P))
```

Answer:

---

**Q56. Predict the output:**

```
class Base:
    def f(self):
        return "B"

class Child(Base):
    def f(self):
        return super().f() + "C"

print(Child().f())
```

Answer:

---

**Q57. Predict the output:**

```

class A:
    def __init__(self, **kwargs):
        self.a = 1
        super().__init__(**kwargs)

class B(A):
    def __init__(self, **kwargs):
        self.b = 2
        super().__init__(**kwargs)

class C:
    def __init__(self, **kwargs):
        self.c = 3
        super().__init__(**kwargs)

class D(B, C):
    pass

d = D()
print(d.a, d.b, d.c)

```

Answer:

---

**Q58. Predict the output:**

```

class Pay:
    def run(self, amount):
        if amount <= 0:
            raise ValueError("bad")
        return "ok"

print(Pay().run(1))

```

Answer:

---

**Q59. Predict the output:**

```

class Pay:
    def run(self, amount):
        if amount <= 0:
            raise ValueError("bad")
        return "ok"

try:
    print(Pay().run(0))
except ValueError as e:
    print(e)

```

Answer:

---

**Q60. Predict the output:**

```

class TimeMixin:
    def touch(self):
        self.updated = True

class User(TimeMixin):
    pass

u = User()
print(hasattr(u, "updated"))
u.touch()
print(hasattr(u, "updated"))

```

Answer:

---

**Q61. Predict the output:**

```
class A:
    logs = []

a1 = A()
a2 = A()
a1.logs.append("x")
print(a1.logs, a2.logs)
```

Answer:

---

**Q62. Predict the output:**

```
class A:
    def __init__(self):
        self.logs = []

a1 = A()
a2 = A()
a1.logs.append("x")
print(a1.logs, a2.logs)
```

Answer:

---

**Q63. Predict the output:**

```
class Report:
    def build(self):
        raise NotImplementedError

class Csv(Report):
    def build(self):
        return "csv"

print(Csv().build())
```

Answer:

---

**Q64. Predict the output:**

```
class Report:
    def build(self):
        return "base"

class Csv(Report):
    pass

print(Csv().build())
```

Answer:

---

**Q65. Predict the output:**

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

v = Vector(1, 2) + Vector(3, 4)
print(v.x, v.y)
```

Answer:

---

**Q66. Predict the output:**

```
from functools import singledispatch

@singledispatch
def show(v):
    return str(v)

@show.register
def _(v: int):
    return f"i:{v}"

print(show(5))
print(show("x"))
```

Answer:

---

**Q67. Predict the output:**

```
class FileStore:
    def save(self, x):
        print("FILE", x)

def backup(store, x):
    store.save(x)

backup(FileStore(), "cfg")
```

Answer:

---

**Q68. Predict the output:**

```
class Parent:
    def __init__(self):
        self.__v = 10

class Child(Parent):
    def check(self):
        return hasattr(self, "_Parent__v")

print(Child().check())
```

Answer:

---

**Q69. Predict the output:**

```
class A:
    def who(self):
        return "A"

class B(A):
    def who(self):
        return "B"

class C(A):
    def who(self):
        return "C"

class D(C, B):
    pass

print(D().who())
```

Answer:

---

**Q70. Predict the output:**

```
class A: pass
class B(A): pass
class C(A): pass
class D(C, B): pass
print([x.__name__ for x in D.mro()])
```

Answer:

---

**Q71. Predict the output:**

```
class Repo:
    def save(self, u):
        print("saved", u)

class UserService:
    def __init__(self, repo):
        self.repo = repo

    def create(self, u):
        self.repo.save(u)

UserService(Repo()).create("neo")
```

Answer:

---

**Q72. Predict the output:**

```
class A:
    def g(self):
        return "A"

class B(A):
    def g(self):
        return "B" + super().g()

class C(B):
    def g(self):
        return "C" + super().g()

print(C().g())
```

Answer:

---

**Q73. Predict the output:**

```
class Formatter:
    def fmt(self, x):
        return {"v": x}

class TextFormatter(Formatter):
    def fmt(self, x):
        return f"v={x}"

print(TextFormatter().fmt(7))
```

Answer:

---

**Q74. Predict the output:**

```
class Parent:
    @property
    def code(self):
        return "P-01"

class Child(Parent):
    pass

print(Child().code)
```

Answer:

---

**Q75. Predict the output:**

```
class Parent:
    role = "parent"

class Child(Parent):
    role = "child"

print(Parent.role, Child.role)
```

Answer:

---

**Q76. Predict the output:**

```
class A:
    @staticmethod
    def ping():
        return "ok"

class B(A):
    pass

print(B.ping())
```

Answer:

---

**Q77. Predict the output:**

```
class A:
    name = "A"
    @classmethod
    def who(cls):
        return cls.name

class B(A):
    name = "B"

print(B.who())
```

Answer:

---

**Q78. Predict the output:**

```
from abc import ABC, abstractmethod

class Base(ABC):
    @abstractmethod
    def x(self):
        pass
    def y(self):
        return "y"

class Child(Base):
    def x(self):
        return "x"

print(Child().x(), Child().y())
```

Answer:

---

**Q79. Predict the output:**

```
class X:
    def p(self):
        return "X"

class Y:
    def p(self):
        return "Y"

class Z(X, Y):
    pass

print(Z().p())
```

Answer:

---

**Q80. Predict the output:**

```
class A:
    def p(self):
        print("A")

class B(A):
    def p(self):
        print("B")
        super().p()

class C(A):
    def p(self):
        print("C")
        super().p()

class D(B, C):
    pass

D().p()
```

Answer:

---

**Q81. Predict the output:**

```
class Auth:
    def login(self, user):
        if not user:
            raise ValueError("user missing")
        return "ok"

try:
    print(Auth().login(""))
except ValueError as e:
    print(e)
```

Answer:

---

**Q82. Predict the output:**

```

class Slack:
    def send(self, m):
        print("SLACK", m)

class Mail:
    def send(self, m):
        print("MAIL", m)

class Notifier:
    def __init__(self, channel):
        self.channel = channel

    def fire(self, m):
        self.channel.send(m)

Notifier(Slack()).fire("build")
Notifier(Mail()).fire("release")

```

Answer:

---

**Q83. Predict the output:**

```

class Payment:
    def pay(self, amt):
        if amt <= 0:
            raise ValueError("invalid")
        print("paid", amt)

class StrictPayment(Payment):
    def pay(self, amt):
        if amt < 100:
            raise ValueError(">=100 only")
        print("strict", amt)

try:
    StrictPayment().pay(50)
except ValueError as e:
    print(e)

```

Answer:

---

**Q84. Predict the output:**

```

class TaxIN:
    def calc(self, x):
        return x * 0.18

class TaxUS:
    def calc(self, x):
        return x * 0.07

class Invoice:
    def __init__(self, tax):
        self.tax = tax

    def total(self, x):
        return x + self.tax.calc(x)

print(round(Invoice(TaxIN()).total(100), 2))
print(round(Invoice(TaxUS()).total(100), 2))

```

Answer:

---

**Q85. Predict the output:**

```
REGISTRY = {}

class Csv:
    def export(self):
        return "csv"

REGISTRY["csv"] = Csv
print(REGISTRY["csv"]().export())
```

Answer:

---

**Q86. Predict the output:**

```
class Retry2:
    def should_retry(self, attempt):
        return attempt < 2

print(Retry2().should_retry(1), Retry2().should_retry(2))
```

Answer:

---

**Q87. Predict the output:**

```
class Client:
    def request(self):
        return {"ok": True}

class Service:
    def __init__(self, client):
        self.client = client

    def run(self):
        return self.client.request()["ok"]

print(Service(Client()).run())
```

Answer:

---

**Q88. Predict the output:**

```
class CacheMixin:
    def mark(self):
        self.cached = True

class Item(CacheMixin):
    pass

i = Item()
i.mark()
print(i.cached)
```

Answer:

---

**Q89. Predict the output:**

```
class A:
    def m(self):
        return "A"

class B:
    def m(self):
        return "B"

class C(B, A):
    pass

print(C().m())
```

Answer:

---

**Q90. Predict the output:**

```
class A: pass
class B: pass
class C(B, A): pass
print(C.__mro__[1].__name__, C.__mro__[2].__name__)
```

Answer:

---

**Q91. Predict the output:**

```
class A:
    def g(self):
        return "A"

class B(A):
    def g(self):
        return super().g() + "B"

class C(B):
    def g(self):
        return super().g() + "C"

print(C().g())
```

Answer:

---

**Q92. Predict the output:**

```
from abc import ABC, abstractmethod

class N(ABC):
    @abstractmethod
    def send(self): ...

class Bad(N):
    pass

try:
    Bad()
except TypeError as e:
    print("TypeError")
```

Answer:

---

**Q93. Predict the output:**

```
class Parent:
    def __init__(self):
        self.__x = 11

p = Parent()
print(hasattr(p, "__x"), hasattr(p, "_Parent__x"))
```

Answer:

---

**Q94. Predict the output:**

```

class FakeLogger:
    def __init__(self):
        self.messages = []
    def log(self, m):
        self.messages.append(m)

class Billing:
    def __init__(self, logger):
        self.logger = logger
    def charge(self, a):
        self.logger.log(f"charge:{a}")

f = FakeLogger()
Billing(f).charge(300)
print(f.messages)

```

Answer:

**Q95. Predict the output:**

```

class M1:
    def x(self):
        return "M1"

class M2:
    def x(self):
        return "M2"

class C(M1, M2):
    pass

print(C().x())

```

Answer:

## Section C: Quick Refactor (96-100)

**Q96. Refactor this bad code (wrong inheritance for HAS-A):**

Refactor in the right-half area.

| BAD Code                                                                                                              | Refactor Here |
|-----------------------------------------------------------------------------------------------------------------------|---------------|
| <pre> class UserService(Database):     def create_user(self, u):         self.connect()         self.insert(u) </pre> | <div></div>   |

**Task:** Refactor to composition. | Keep behavior equivalent. | Use clean naming.

**Q97. Refactor this bad code (conditional explosion):**

Refactor in the right-half area.

| BAD Code                                                                                                                                                                                                                                                           | Refactor Here |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <pre> def process_payment(mode, amount):     if mode == "card":         return amount * 1.02     elif mode == "upi":         return amount * 1.00     elif mode == "wallet":         return amount * 0.98     else:         raise ValueError("unsupported") </pre> | <div></div>   |

**Task:** Replace with polymorphic design. | No `if/elif` in core processing flow.

Q98. Refactor this bad code (LSP violation risk):

Refactor in the right-half area.

| BAD Code                                                                                                                                                                                                                                                | Refactor Here |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <pre>class Payment:     def pay(self, amount):         if amount &lt;= 0:             raise ValueError("invalid")  class SpecialPayment(Payment):     def pay(self, amount):         if amount &lt; 1000:             raise ValueError("invalid")</pre> |               |

Task: Make child behavior LSP-safe. | Keep business rule extensible.

Q99. Refactor this bad code (hardcoded dependency + poor testability):

Refactor in the right-half area.

| BAD Code                                                                                                                                                       | Refactor Here |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <pre>class OrderService:     def __init__(self):         self.logger = ConsoleLogger()      def place(self, order_id):         self.logger.log(order_id)</pre> |               |

Task: Use dependency injection. | Depend on abstraction, not concrete logger.

Q100. Refactor this bad code (shared mutable class state bug):

Refactor in the right-half area.

| BAD Code                                                                                  | Refactor Here |
|-------------------------------------------------------------------------------------------|---------------|
| <pre>class Cart:     items = []      def add(self, x):         self.items.append(x)</pre> |               |

Task: Fix shared mutable bug. | Keep API simple and safe.